

Experiment 1 by Amrut Anand - Roll: 24011101003

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis

- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation

- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

GitHub repository: https://github.com/the-future-codemaster/dl_lab.git

7. Experimental Results

Task 1: Dataset Exploration Output

First Five Samples

Index	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Dataset Overview

Property	Value
Total Rows	1372
Total Columns	5
Missing Values (All Features)	0

Descriptive Statistics

Feature	Count	Mean	Std Dev	Min	25%	50%	75%	Max
Variance	1372.0	0.4337	2.8428	-7.0421	-1.7730	0.4962	2.8215	6.8248
Skewness	1372.0	1.9224	5.8690	-13.7731	-1.7082	2.3197	6.8146	12.9516
Curtosis	1372.0	1.3976	4.3100	-5.2861	-1.5750	0.6166	3.1793	17.9274
Entropy	1372.0	-1.1917	2.1010	-8.5482	-2.4135	-0.5867	0.3948	2.4495
Class	1372.0	0.4446	0.4971	0.0000	0.0000	0.0000	1.0000	1.0000

Task 3: Data Preprocessing Output

First Three Rows of Scaled Features (Z-Score Normalization)

Index	Variance	Skewness	Curtosis	Entropy
0	1.1218	1.1495	-0.9760	0.3546
1	1.4471	1.0645	-0.8950	-0.1288
2	1.2078	-0.7774	0.1222	0.6181

Split Partition Overview

Split Partition	Sample Size
Training Set	1097
Testing Set	275

Task 6: Model Evaluation Output

Evaluation Confusion Matrix

	Predicted Authentic (0)	Predicted Forged (1)
Actual Authentic (0)	True Negative (TN): 152	False Positive (FP): 1
Actual Forged (1)	False Negative (FN): 2	True Positive (TP): 120

Classification Metrics

Accuracy	Precision	Recall	F1-Score
0.9891	0.9917	0.9836	0.9877

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80% / 20% (1097 Train, 275 Test)
Learning Rate	0.01
Epochs	15
Final Weights (Variance, Skewness)	-0.1439, -0.1603
Final Bias	-0.0700
Accuracy	0.9891
Precision	0.9917
Recall	0.9836
F1-score	0.9877

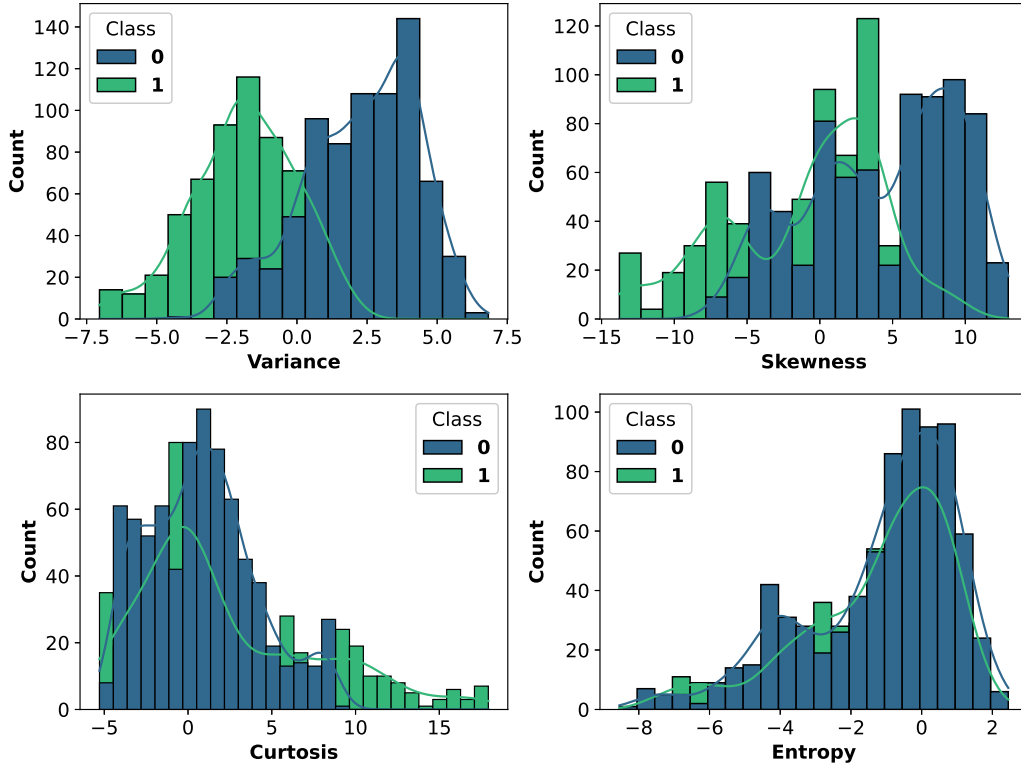
Epoch-wise Learning (Full 15 Epochs)

Epoch	Errors	Weight 1 (Var)	Weight 2 (Skew)	Bias
1	57	-0.0623	-0.0697	-0.0300
2	32	-0.0851	-0.0860	-0.0500
3	31	-0.1048	-0.1132	-0.0400
4	28	-0.0950	-0.1159	-0.0600
5	26	-0.1032	-0.1403	-0.0400
6	22	-0.1115	-0.1438	-0.0600
7	22	-0.1162	-0.1511	-0.0600
8	21	-0.1106	-0.1509	-0.0700
9	23	-0.1307	-0.1569	-0.0600
10	23	-0.1285	-0.1643	-0.0500
11	18	-0.1365	-0.1493	-0.0700
12	24	-0.1320	-0.1651	-0.0700
13	18	-0.1314	-0.1638	-0.0700
14	17	-0.1349	-0.1646	-0.0800
15	11	-0.1439	-0.1603	-0.0700

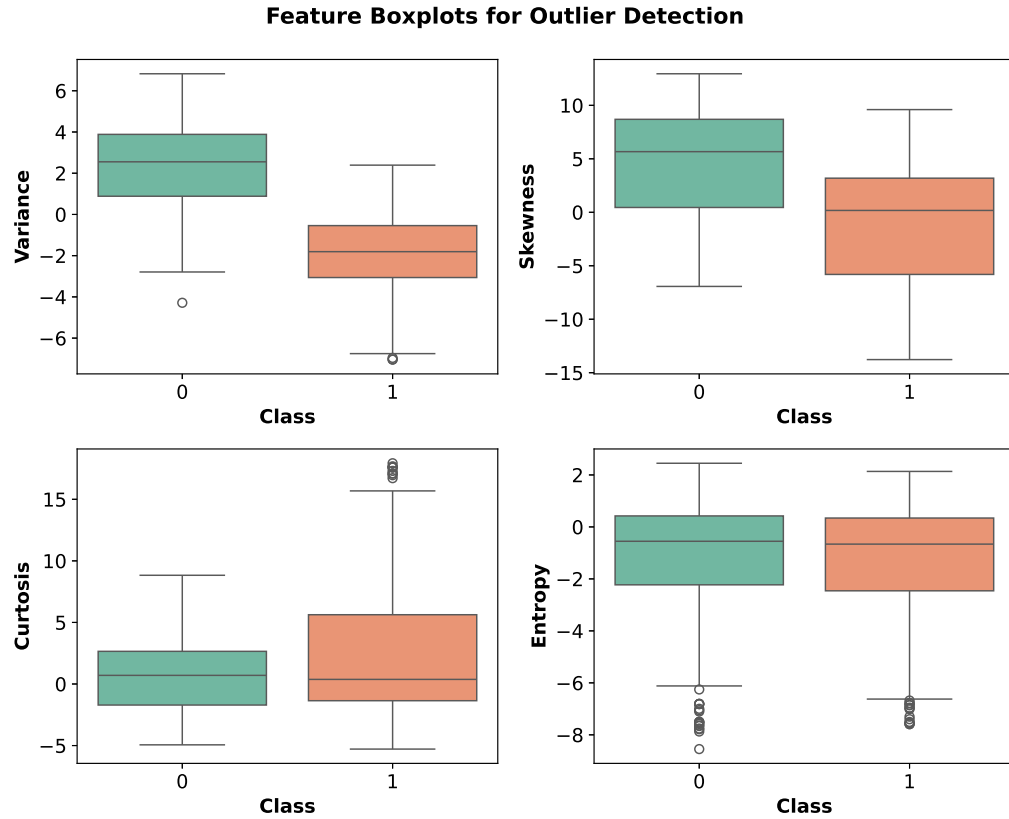
9. Experimental Plots & Interpretations

Exploratory Data Analysis

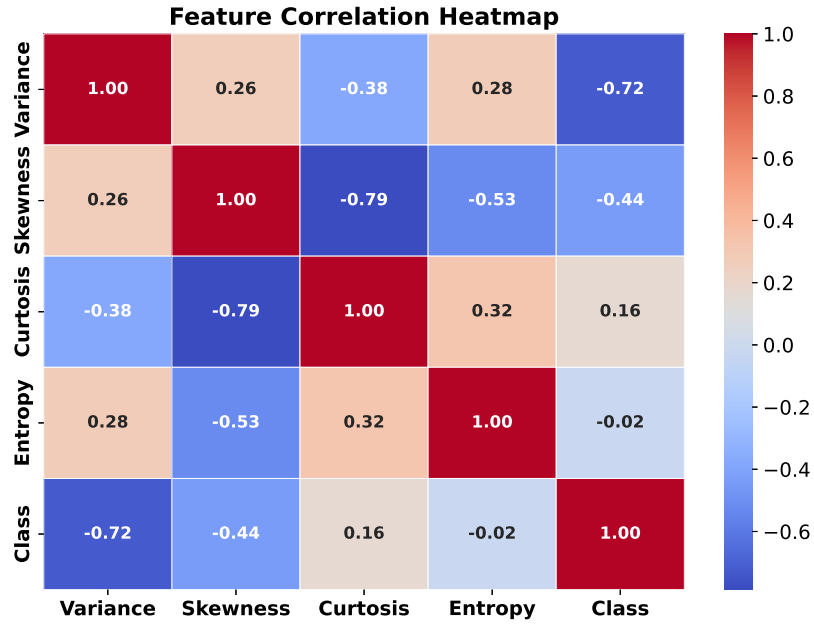
Feature Histograms



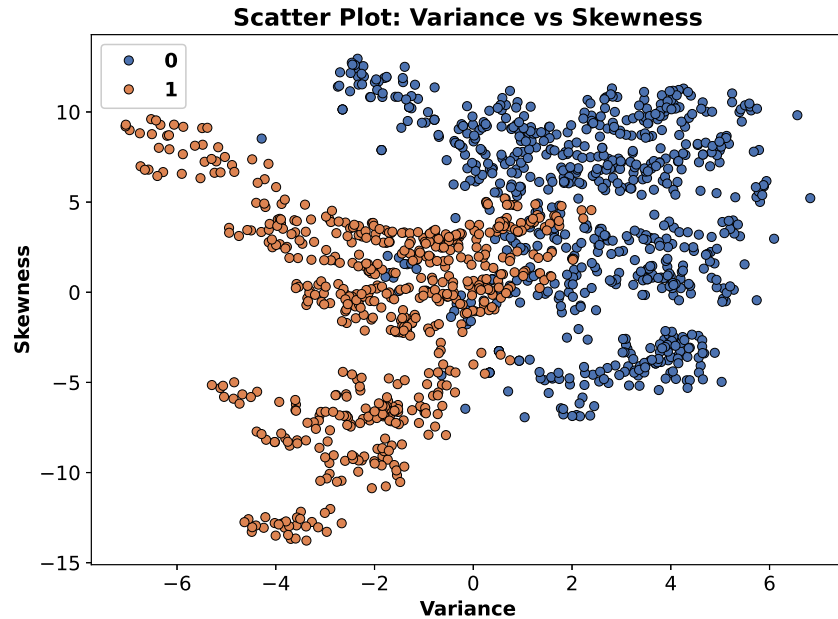
Interpretation: Across each class, the histograms shows differing distributions across classes. Variance has a bimodal tendency which helps in the initial separation. Curtosis shows the presence of extreme values due to a heavy right skew.



Interpretation: The boxplots confirm significant extreme outliers visually and most predominantly within Curtosis and Skewness. Hence the application of the Z-score standardisation before training is shown.



Interpretation: The heatmap shows pairwise linear relationships. Also there is a strong negative correlation between Skewness and Kurtosis implying some redundancy in the wavelet feature space mapping.



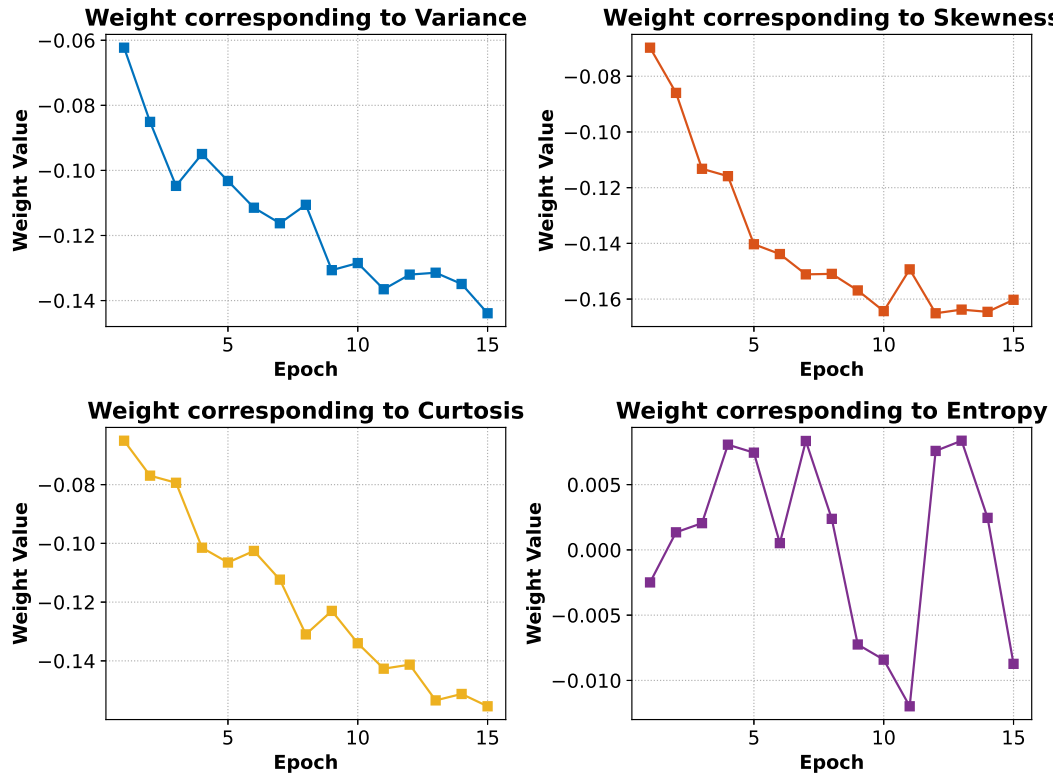
Interpretation: The scatter plot of Variance against Skewness reveals geometric clusters which approximates that the feature space is linearly separable using Single Layer Perceptron.

Model Training & Evaluation

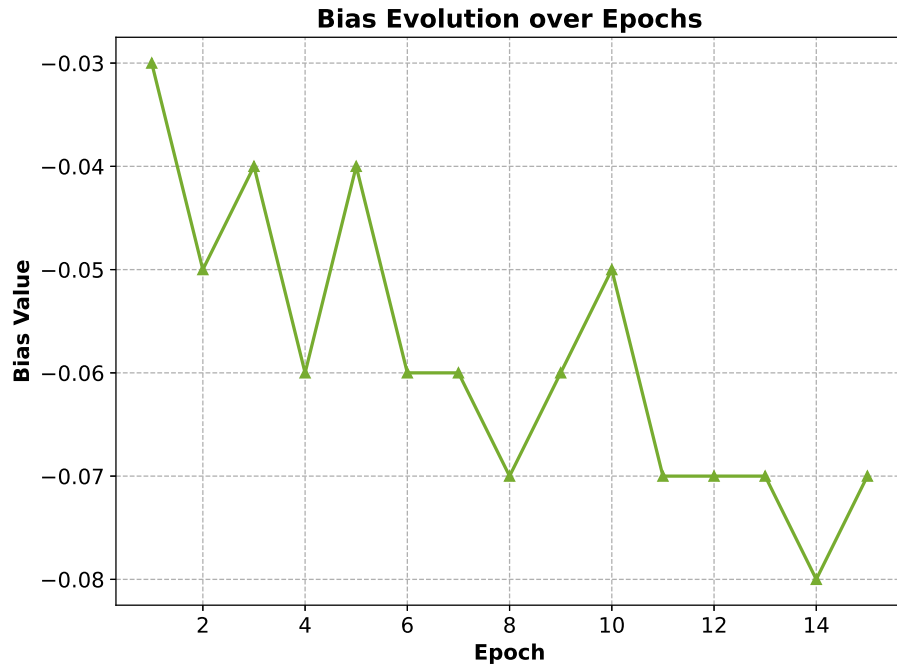


Interpretation: The training error vs epoch graph shows a rapid drop in the number of misclassified samples. We can confirm linear separability by observing the graph converging to a almost steady state within the first few epochs.

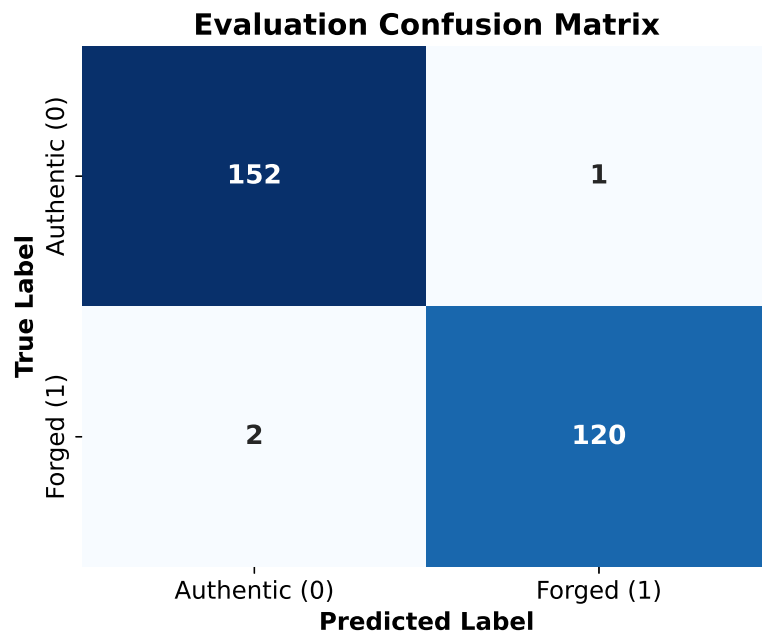
Evolution of Learned Weights over Epochs



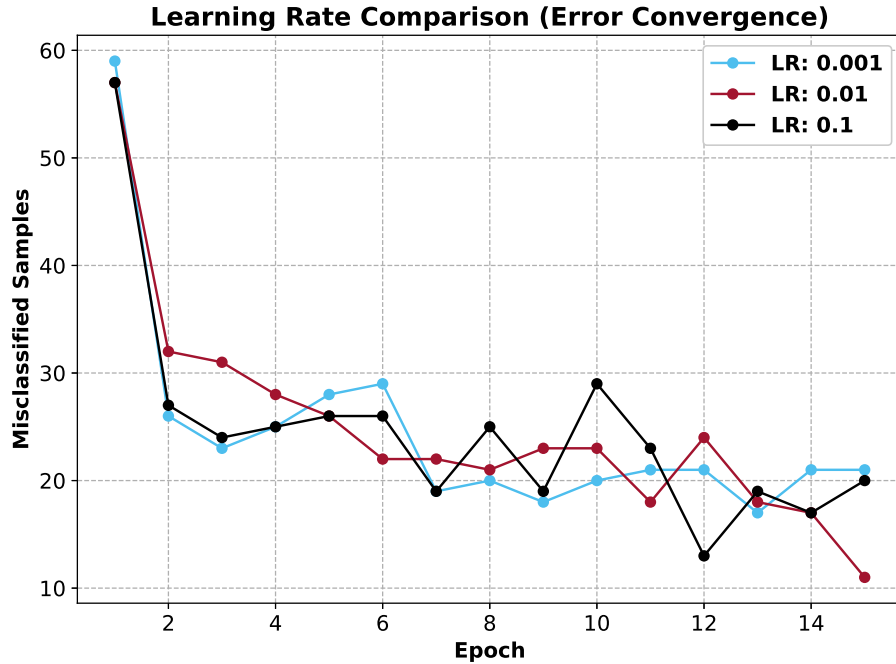
Interpretation: The indication that the optimal separating hyperplane vector $\mathbf{w}$ has been found is when the the error gradient approaches zero. This is when the weights initially go through aggressive spatial adjustments before then stabilising.



Interpretation: The bias term changes and plateaus according to the the weights. These changes highlight its role in making the decision boundary segregate the feature distributions after scaling.



Interpretation: The matrix validates generalisation on unseen test data by showing 120 True Positives and 152 True Negatives. There are only 1 or 2 false classifications so we get near-perfect precision and recall metrics.



Interpretation: We get the balance of gradient optimisation by comparing the convergence across learning rates. The larger rate ($\eta = 0.1$) has more oscillations overshooting, while the smaller rates has smoother convergence by getting lower misclassified errors toward the global minimum.

10. Discussion

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.

The step function gives a hard 0 or 1 with a sudden jump in the middle at origin. On the other hand, sigmoid function gives a continuous S-curve between 0 and 1. Backpropagation in modern deep learning is relied on heavily to train networks. They also require calculating derivatives to update weights. The problem starts when the step function's derivative is zero everywhere and also undefined at the jump. When we get a gradient of zero, then network can't learn anything. But since Sigmoid is differentiable everywhere, the errors are allowed to flow backward and adjust gradually in the network.

2. Compare the implementation with Scikit-learn's Perceptron.

Our implementation outperformed Scikit-learn's version by getting a 98.91% accuracy while sklearn's got 97.45%. This is because Scikit-learn's **Perceptron** uses a highly optimised Stochastic Gradient Descent (SGD) which also has an early stopping criteria a.k.a tolerance. It also shuffles data. Whereas our code simply forces the model to train for the full 15 epochs. It iterates in the same order every time. The dataset is simple and linearly separable and while sklearn may have stopped training early to prevent overfitting, our brute-force approach happened to find a better decision boundary; for this specific train/test split.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

The three different learning rates gives us three different convergence behaviours. 0.001: though the error decreases steadily, it converges slowly and plateaus at around 20 misclassified samples. 0.1: this lr converges initially faster but then it shows some fluctuations during the training which shows less stable updates. 0.01: it has some fluctuations but it has the lowest final error (11 misclassified samples) within the three learning rates. Overall, 0.01 is the best final performance in this experiment. 0.001 is the most stable and 0.1 converges quickly but has more oscillations.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.

A single layer perceptron is basically like a linear classifier i.e. it can draw a straight line to separate data points. If we plot the inputs for a XOR gate, the outputs of '1' are opposite from each other and the outputs of '0' are also diagonally opposite from each other. We cannot draw a straight line across a graph to separate the 1's from the 0's. We see that the data is not linearly separable and so the perceptron fails. Instead we need to draw multiple lines.

5. Study the effect of feature normalization on model convergence.

Avoiding normalisation will give us features where the larger numbers will completely dominate the weight updates. The model will focus mostly on those larger features thereby ignoring the smaller ones. This will cause the training process to either extend or completely fail. The Z-Score ensures every feature gives us a mean of 0 and the standard deviation of 1. This balances and gives the error surface a much rounder shape. Therefore the gradient descent algorithm can take more direct and efficient steps toward the minimum, making convergence significantly faster.

11. Conclusion

Thus a single layer perceptron was successfully implemented from scratch to classify between the authentic and the forged banknotes. We thoroughly analysed the dataset, then we pre-processed the features with the Z-score normalisation and finally tuned the learning rate. Following these resulted a test accuracy of 98.91%, showing the effectiveness on linearly separable data.

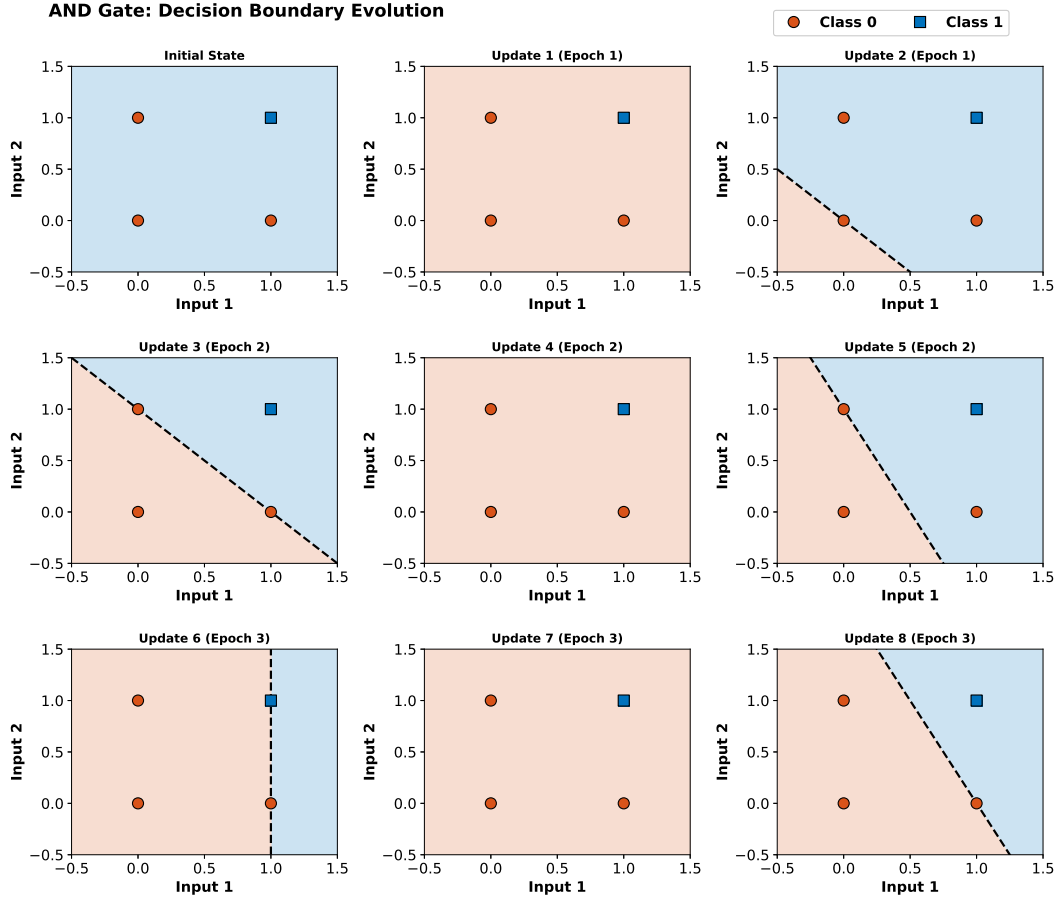
12. Additional Tasks

Task 1: Single Layer Perceptron for Logic Gates (AND, OR, NOT)

AND Gate Training History

Update	Weights $[w_1, w_2]$	Bias	Triggering Input
1	[0.0, 0.0]	-0.10	[0, 0]
2	[0.1, 0.1]	0.00	[1, 1]
3	[0.1, 0.1]	-0.10	[0, 0]
4	[0.1, 0.0]	-0.20	[0, 1]
5	[0.2, 0.1]	-0.10	[1, 1]
6	[0.2, 0.0]	-0.20	[0, 1]
7	[0.1, 0.0]	-0.30	[1, 0]
8	[0.2, 0.1]	-0.20	[1, 1]

Note: Model converged at epoch 4.

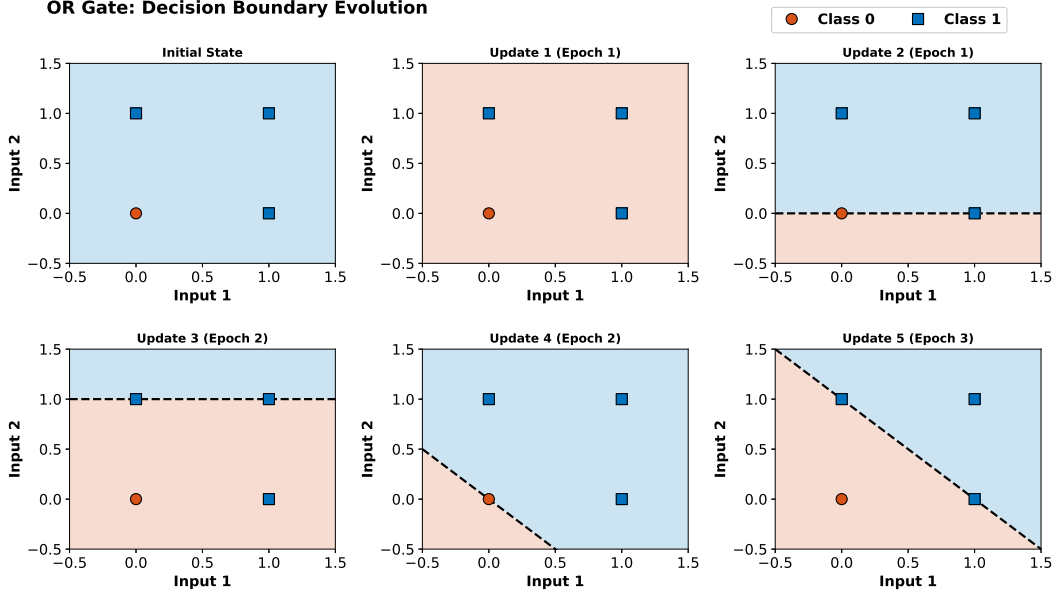


Interpretation: When both inputs are 1, we get an output of 1 for an AND gate. The changing decision boundary is shown for the 8 updates across 3 epochs. The linear separation of the positive and the negative class using a single straight line was successfully done by the Single-Layer Perceptron.

OR Gate Training History

Update	Weights $[w_1, w_2]$	Bias	Triggering Input
1	$[0.0, 0.0]$	-0.10	$[0, 0]$
2	$[0.0, 0.1]$	0.00	$[0, 1]$
3	$[0.0, 0.1]$	-0.10	$[0, 0]$
4	$[0.1, 0.1]$	0.00	$[1, 0]$
5	$[0.1, 0.1]$	-0.10	$[0, 0]$

Note: Model converged at epoch 4.

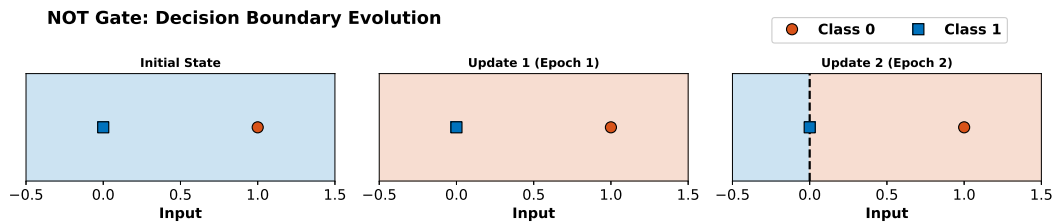


Interpretation: When any input is 1, we get an output of 1 for an OR gate. The changing decision boundary is shown for the 5 updates across 3 epochs. It successfully separates $[0,0]$ from the rest. We can see that the linear separation of the positive and negative class was done by the Single-Layer Perceptron using an optimal hyperplane successfully.

NOT Gate Training History

Update	Weight $[w]$	Bias	Triggering Input
1	$[-0.1]$	-0.10	$[1]$
2	$[-0.1]$	0.00	$[0]$

Note: Model converged at epoch 3.



Interpretation: The NOT gate has only 1D input. It was correctly separated in 2 updates over 3 epochs. Therefore the Single-Layer Perceptron successfully identified the 2 classes.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.